

Bachelorthesis

Integration von SAPL in den Linux-Kernel

Christian Koch

3. Juni 2026

Inhaltsverzeichnis

1	Einleitung	3
1.1	Motivation	3
1.2	Zielsetzung	3
1.3	Aufbau der Arbeit	3
2	Grundlagen	4
2.1	Linux Security Modules	4
2.2	Attribut Stream Based Access Control	4
2.3	SAPL	4
3	Anforderungsanalyse	5
3.1	Funktionale Anforderungen	5
3.2	Nichtfunktionale Anforderungen	6
4	Konzeption	8
4.1	Architektur	8
4.2	Datenfluss	8
4.2.1	Policy-Datenfluss	8
4.2.2	Datenfluss dynamischer Attribute	9
4.2.3	Zugriffsentscheidungen im Kernel	9
4.2.4	Monitoring bestehender Zugriffe	9
4.3	Komponenten	10
4.3.1	Policies	10
4.3.2	Policyverwaltung	10
4.3.3	Monitor	10
4.3.4	Datenobjekte im Userspace	10
4.3.5	Datenobjekte im Kernel	11
4.3.6	LSM	11
4.3.7	Administrationstool	11
4.4	Entwurfsentscheidungen	11
5	Entwurf und Umsetzung	12
5.1	Komponente 1	12
5.2	Komponente 2	12
5.3	Komponente 3	12
5.4	Implementierungsdetails	12
6	Evaluation	13
6.1	Testumgebung	13
6.2	Testsznarien	13
6.3	Ergebnisse	13
7	Zusammenfassung und Ausblick	14

1 Einleitung

1.1 Motivation

Die Sicherheit von Linux-Systemen ist ein fundamentales Anliegen in der heutigen IT-Landschaft. Das Linux Security Module Framework stellt Mechanismen für verschiedene Sicherheitsprüfungen mittels Attribut Based Access Control im Linuxkernel dar [?]. Anhand von Hooks, welche an den Aufruf einer Ressource gebunden sind, lassen sich mittels eines LSMs Zugriffsentscheidungen treffen. Dieser Mechanismus stellt jedoch nur eine einmalige Zugriffskontrolle dar. Wenn auf eine Ressource dauerhaft zugegriffen wird, kann anhand eines Hooks lediglich einmal geprüft werden, der Zugriff findet dann, auch bei geänderten Bedingungen weiterhin statt.

1.2 Zielsetzung

Zielsetzung der Arbeit ist ein Prototyp, anhand dessen eine Zugriffskontrolle mittels Attribut Stream based Access Control im Linuxkernel stattfindet. Hier sollen insbesondere Stream-Attribute, wie z.B. die Zeit und von außen in Echtzeit in den Kernel gestreamte Attribute, zur Entscheidung herangezogen werden. Ziel ist hier die Implementierung eines Prototypen, der Policies einliest und in den Kernel und Zugriffsentscheidungen anhand dieser Policies trifft. Weiterhin sollen diese Policies auch überwacht und bei sich ändernden Bedingungen notwendigerweise auch mittels eines Enforcements umgesetzt werden.

1.3 Aufbau der Arbeit

2 Grundlagen

2.1 Linux Security Modules

2.2 Attribut Stream Based Access Control

2.3 SAPL

3 Anforderungsanalyse

In diesem Kapitel werden die Anforderungen an den zu entwickelnden Prototypen beschrieben. Die Anforderungen ergeben sich aus der Zielsetzung, SAPL-basierte Zugriffskontrolle im Linux-Kernel zu untersuchen und prototypisch umzusetzen. Dabei wird zwischen funktionalen Anforderungen, die das konkrete Verhalten des Systems beschreiben, und nichtfunktionalen Anforderungen, die qualitative Eigenschaften des Systems festlegen, unterschieden.

3.1 Funktionale Anforderungen

Die funktionalen Anforderungen beschreiben, welche Aufgaben der Prototyp erfüllen muss. Sie beziehen sich insbesondere auf die Verarbeitung von Policies, die Kommunikation zwischen Userspace und Kernelspace sowie die Durchsetzung von Zugriffsentscheidungen.

FA-01 Policy-Einlesen: Der Prototyp muss SAPL-Policies aus einer definierten Quelle im Userspace einlesen können.

Akzeptanzkriterium: Eine im Policy-Verzeichnis abgelegte Policy wird erkannt und für die weitere Verarbeitung bereitgestellt.

FA-02 Policy-Verwaltung: Der Prototyp muss Änderungen an Policies erkennen und aktualisierte Policies in die Zugriffskontrolle übernehmen können.

Akzeptanzkriterium: Wird eine Policy geändert, hinzugefügt oder entfernt, wird diese Änderung vom System verarbeitet.

FA-03 Kommunikation zwischen Userspace und Kernelspace: Der Prototyp muss eine Schnittstelle bereitstellen, über die Informationen zwischen Userspace und Kernelspace ausgetauscht werden können.

Akzeptanzkriterium: Der Userspace kann dem Kernel entscheidungsrelevante Informationen übermitteln.

FA-04 Zugriffskontrolle über LSM-Hooks: Der Prototyp muss sicherheitsrelevante Operationen über geeignete Linux-Security-Module-Hooks abfangen können.

Akzeptanzkriterium: Mindestens ein relevanter Zugriff auf eine Ressource wird durch einen LSM-Hook kontrolliert.

FA-05 Entscheidungsfindung anhand von Policies: Der Prototyp muss Zugriffsentscheidungen auf Grundlage der geladenen Policies treffen können.

Akzeptanzkriterium: Ein Zugriff wird abhängig von einer Policy erlaubt oder verweigert.

FA-06 Unterstützung dynamischer Attribute: Der Prototyp muss dynamische Attribute, beispielsweise Zeitinformationen oder externe Zustandsinformationen, in Zugriffsentscheidungen einbeziehen können.

Akzeptanzkriterium: Eine Änderung eines dynamischen Attributs kann zu einer veränderten Zugriffsentscheidung führen.

FA-07 Überwachung bestehender Zugriffe: Der Prototyp muss bestehende Zugriffe überwachen können, wenn sich entscheidungsrelevante Attribute zur Laufzeit ändern.

Akzeptanzkriterium: Das System erkennt, wenn eine ursprünglich erlaubte Zugriffssituation aufgrund geänderter Attribute nicht mehr zulässig ist.

FA-08 Enforcement bei geänderten Bedingungen: Der Prototyp muss bei geänderten Bedingungen eine zuvor erlaubte Zugriffssituation einschränken oder beenden können, soweit dies im Rahmen des Prototyps technisch vorgesehen ist.

Akzeptanzkriterium: Ein Zugriff wird nach einer relevanten Attributänderung entsprechend der Policy unterbunden oder eingeschränkt.

FA-09 Administrationsschnittstelle: Der Prototyp soll eine einfache Administrationschnittstelle bereitstellen, über die Policies oder Systemzustände eingesehen beziehungsweise gesteuert werden können.

Akzeptanzkriterium: Ein Administrator kann zentrale Informationen zum Zustand des Prototyps abrufen.

FA-10 Gültige Policy-Generationen: Der Prototyp muss sicherstellen, dass nur erfolgreich erzeugte und validierte Generationen von Policies für Zugriffsentscheidungen verwendet werden. Fehlerhafte, unvollständige oder nicht erfolgreich verarbeitete Policy-Generationen dürfen nicht als aktive Entscheidungsgrundlage gelten.

Akzeptanzkriterium: Eine neue Policy-Generation wird erst dann aktiviert, wenn sie vollständig verarbeitet und erfolgreich validiert wurde. Andernfalls bleibt die zuletzt gültige Policy-Generation aktiv.

3.2 Nichtfunktionale Anforderungen

Die nichtfunktionalen Anforderungen beschreiben qualitative Eigenschaften des Prototyps. Sie sind insbesondere relevant, da die Umsetzung im Kontext des Linux-Kernels erfolgt und daher Stabilität, Sicherheit und Nachvollziehbarkeit eine besondere Rolle spielen.

NFA-01 Stabilität: Der Prototyp darf die Stabilität des Linux-Kernels nicht unnötig gefährden.

Akzeptanzkriterium: Fehlerhafte oder ungültige Eingaben aus dem Userspace führen nicht zu einem Kernel-Absturz.

NFA-02 Sicherheit: Die Kommunikation zwischen Userspace und Kernelspace muss so gestaltet sein, dass ungültige oder manipulierte Eingaben geprüft werden.

Akzeptanzkriterium: Eingaben werden validiert, bevor sie im Kernel für Zugriffsentscheidungen verwendet werden.

NFA-03 Nachvollziehbarkeit: Relevante Entscheidungen und Zustandsänderungen sollen nachvollziehbar protokolliert werden.

Akzeptanzkriterium: Für zentrale Zugriffsentscheidungen ist erkennbar, welche Policy oder welcher Zustand die Entscheidung beeinflusst hat.

NFA-04 Wartbarkeit: Die Implementierung soll modular aufgebaut sein, sodass Userspace-Komponenten, Kernel-Komponenten und Kommunikationsschnittstelle getrennt betrachtet werden können.

Akzeptanzkriterium: Die zentralen Komponenten sind im Quellcode klar voneinander getrennt.

NFA-05 Erweiterbarkeit: Der Prototyp soll so entworfen werden, dass weitere Attribute, Policies oder LSM-Hooks ergänzt werden können.

Akzeptanzkriterium: Die Architektur erlaubt die Ergänzung weiterer Attribute oder Hooks ohne grundlegende Umstrukturierung.

NFA-06 Performance: Die zusätzlichen Prüfungen durch den Prototyp sollen den kontrollierten Zugriff nicht unverhältnismäßig verlangsamen.

Akzeptanzkriterium: Die Auswirkungen auf die Ausführungszeit werden in der Evaluation betrachtet.

NFA-07 Begrenzter Kernel-Anteil: Komplexe Policy-Logik soll möglichst im Userspace verbleiben, um die Komplexität im Kernel zu reduzieren.

Akzeptanzkriterium: Der Kernel übernimmt vor allem die Durchsetzung der Entscheidung, während komplexere Verarbeitungsschritte im Userspace stattfinden.

NFA-08 Reproduzierbarkeit: Die Evaluation des Prototyps soll unter dokumentierten Bedingungen wiederholbar sein.

Akzeptanzkriterium: Testumgebung, Policies und Testszenarien werden so beschrieben, dass die Ergebnisse nachvollzogen werden können.

4 Konzeption

4.1 Architektur

Die Architektur des Prototyps ist in zwei zentrale Bereiche unterteilt: den Userspace und den Kernelspace. Diese Trennung orientiert sich an der grundsätzlichen Struktur von Linux-Systemen und ermöglicht es, komplexe Logik zur Policy-Verarbeitung außerhalb des Kernels auszuführen, während sicherheitskritische Zugriffentscheidungen direkt im Kernel umgesetzt werden.

- **Userspace:** Im Userspace befinden sich die Komponenten zur Verwaltung und Verarbeitung der SAPL-Policies. Dieser unterteilt sich in das Hauptprogramm, einen Attributloader, einen Monitor und die Policyverwaltung. Das Hauptprogramm lädt das Kernelprogramm und startet dieses. Der Attributloader lädt Streamingattribute in den Kernelspace. Die Policyverwaltung liest die SAPL-Policies, parst diese und lädt sie bei in den Kernel. Der Monitor überwacht Dateizugriffe und Socketzugriffe im Userspace und prüft etwaige Policyverletzungen. Mittels einem Enforcement kann der Prozess bei einer Policyverletzung beendet oder der Dateizugriff mittels löschen des FileDescriptors beendet werden.
- **Kernelspace:** Im Kernelspace wird das Linux Security Module integriert. Dieses greift über LSM-Hooks in sicherheitsrelevante Operationen ein und trifft Zugriffentscheidungen auf Basis der vom Userspace bereitgestellten Policies. Hierbei wertet das LSM auch Streamingattribute, welche vom Userspace aus in den Kernel gestreamt werden aus.

Die Kommunikation zwischen Userspace und Kernelspace bildet dabei eine zentrale Schnittstelle des Systems. Sie dient dazu, Policies, Entscheidungen oder Attribute zwischen beiden Bereichen auszutauschen. Dadurch kann die Policy-Logik im Userspace verbleiben, während der Kernel die eigentliche Durchsetzung der Zugriffskontrolle vor Dateizugriff bzw. Socketbind übernimmt. Die nachträgliche Zugriffskontrolle bei schon erlaubtem FileHook ist mit einem LSM nicht möglich, dies wird durch den Monitor überwacht und enforced.

4.2 Datenfluss

Der Datenfluss des Prototyps lässt sich in vier Teilbereiche unterteilen. Diese Teilbereiche beschreiben, wie Policies in das System gelangen, wie dynamische Attribute aktualisiert werden, wie ein Zugriff im Kernel bewertet wird und wie bereits bestehende Zugriffe nachträglich überwacht werden.

4.2.1 Policy-Datenfluss

Policies werden im Userspace in einem dafür vorgesehenen Policy-Verzeichnis abgelegt. Die Policyverwaltung überwacht dieses Verzeichnis und erkennt, wenn Policies hinzugefügt, geändert oder entfernt werden. Nach einer Änderung werden die vorhandenen Policy-Dateien eingelesen und in eine interne Darstellung überführt. Dabei wird geprüft, ob die Policies syntaktisch und semantisch für den Prototyp verarbeitbar sind.

Erst wenn diese Verarbeitung erfolgreich abgeschlossen ist, werden die Policies als neue Policy-Generation in den Kernel übertragen. Die Übertragung erfolgt über eBPF-Maps, die als gemeinsame Datenstruktur zwischen Userspace und Kernelspace dienen. Durch das Generationenmodell wird verhindert, dass der Kernel während einer Zugriffsentscheidung eine nur teilweise aktualisierte Policy-Menge verwendet. Schlägt die Verarbeitung oder Übertragung fehl, bleibt die zuletzt gültige Policy-Generation aktiv.

4.2.2 Datenfluss dynamischer Attribute

Neben den Policies werden dynamische Attribute aus dem Userspace in den Kernelspace übertragen. Dynamische Attribute sind Werte, die sich zur Laufzeit ändern und deshalb regelmäßig aktualisiert werden müssen. Im Prototyp betrifft dies insbesondere Zeitinformationen sowie ein extern bereitgestelltes DEFCON-Level.

Die Userspace-Komponente ermittelt diese Werte beziehungsweise liest sie aus einer definierten Quelle ein. Anschließend werden die aktuellen Werte in dafür vorgesehene eBPF-Maps geschrieben. Der Kernel kann diese Werte bei der Auswertung von Streampolicies lesen und in die Zugriffsentscheidung einbeziehen. Dadurch können Policies formuliert werden, deren Entscheidung nicht nur von statischen Eigenschaften wie Benutzer-ID oder Ressource abhängt, sondern auch von veränderlichen Laufzeitzuständen.

4.2.3 Zugriffsentscheidungen im Kernel

Wenn ein kontrollierter Zugriff stattfindet, wird im Kernel ein LSM-Hook ausgelöst. Bei einem Dateizugriff ist dies beispielsweise der Hook `file_open`, bei einem Socket-Bind der Hook `socket_bind`. Das eBPF-Programm liest aus dem Hook-Kontext die für die Entscheidung relevanten Informationen aus. Dazu gehören unter anderem das Subjekt, die angefragte Ressource und der aktuelle Prozesskontext.

Auf Grundlage dieser Zugriffsdaten werden die zugehörigen Policies im Kernel ausgewertet. Dabei werden nur Policies berücksichtigt, die für den jeweiligen Hook vorgesehen und auf die konkrete Zugriffssituation anwendbar sind. Eine Policy ist beispielsweise nur dann anwendbar, wenn Benutzer, Prozessname und Ressource zu der aktuellen Anfrage passen. Erst wenn eine Policy anwendbar ist, kann sie eine Aussage in Form von *Permit* oder *Deny* treffen.

Die Zwischenergebnisse der Policy-Auswertung werden anschließend zusammengeführt. Das Combining entscheidet, ob der Zugriff insgesamt erlaubt oder verweigert wird. Führt das Ergebnis zu einer Verweigerung, gibt das eBPF-LSM-Programm einen Fehlerwert an den Kernel zurück. Der Kernel unterbindet daraufhin den Zugriff.

4.2.4 Monitoring bestehender Zugriffe

LSM-Hooks prüfen einen Zugriff zu dem Zeitpunkt, an dem das jeweilige Kernelereignis auftritt. Bei streambasierten Policies kann sich die Entscheidungslage jedoch nachträglich ändern, beispielsweise wenn sich die Uhrzeit oder das DEFCON-Level ändert. Ein bereits geöffneter Zugriff wird dadurch nicht automatisch erneut durch denselben LSM-Hook geprüft.

Aus diesem Grund enthält der Prototyp zusätzlich einen Monitor im Userspace. Der Monitor betrachtet regelmäßig aktive Prozesse und deren aktuell verwendete Ressourcen.

Dabei prüft er, ob eine bestehende Zugriffssituation mit den derzeit aktiven Policies vereinbar ist. Wird eine Verletzung erkannt, kann der Monitor ein nachträgliches Enforcement auslösen. Im Fall von Dateizugriffen kann dies beispielsweise bedeuten, dass betroffene File Descriptors eines Prozesses geschlossen werden.

Der Monitor ergänzt damit die kernelbasierte Zugriffskontrolle. Während die LSM-Hooks neue Zugriffe unmittelbar prüfen, adressiert der Monitor das Problem bereits bestehender Zugriffe bei nachträglich geänderten Bedingungen.

4.3 Komponenten

Die Architektur des Prototyps setzt sich aus mehreren Komponenten zusammen, die jeweils eine abgegrenzte Aufgabe innerhalb der Zugriffskontrolle übernehmen. Im Folgenden werden die wichtigsten Komponenten beschrieben.

4.3.1 Policies

Im Userspace befinden sich Policies in einem Verzeichnis, welches von der Policyverwaltung überwacht wird. Es wird zwischen statischen Policies und Streampolicies unterschieden. Die statischen Policies können ohne die Einbeziehung von Streamingattributen evaluiert werden. Die Streampolicies evaluieren in Abhängigkeit von Streamingattributen.

4.3.2 Policyverwaltung

Die Policies werden in Textform und SAPL-Syntax in einem Verzeichnis hinterlegt. Dieses Verzeichnis wird von der Policyverwaltung überwacht und bei einer Änderung in den Kernel geladen. Da es hier unter ReCA COnditions zu Inkonsistenzen kommen kann, werden die Policies nur als atomare Einheit in den Kernel geladen. Dies wird durch ein Generationensystem gewährleistet. Eine neue Generation Policies wird vor der Anwendung und dem Laden in den Kernelspace zuerst im Userspace validiert. Nach erfolgreicher Validierung werden die Policies in den Kernelspace übertragen. Weiterhin wird hierdurch die Fehlertoleranz erhöht, da keine halb geschriebene Policygeneration aktiv ist.

Die Policyverwaltung beinhaltet Gelesene Policy-Dokumente, gepackte Policies, kompilierte Policies, Policygenerationen und aktive und zuletzt gültige Policyversionen.

4.3.3 Monitor

Der Monitor überwacht Prozesse in Userspace und prüft, ob diese mit aktiven Policies in Konflikt stehen. Ist dies der Fall, tritt eine Folge ein. Je nach Filehook, wird z.B. der Prozess aus dem Userspace heraus beendet oder der Filedescriptor gelöscht.

4.3.4 Datenobjekte im Userspace

Weiterhin befinden sich im Userspace Streamingattribute. Diese sind die aktuelle Uhrzeit und ein DEFCON-Level. Diese Attribute werden bei Änderung, bzw. im Fall der Uhrzeit jede Sekunde in den Kernel gestreamt. Der Monitor

4.3.5 Datenobjekte im Kernel

Datenobjekte im Kernel sind Policies. Diese werden nach Validierung aus dem Userspace in den Kernelspace geladen.

4.3.6 LSM

4.3.7 Administrationstool

4.4 Entwurfsentscheidungen

5 Entwurf und Umsetzung

5.1 Komponente 1

5.2 Komponente 2

5.3 Komponente 3

5.4 Implementierungsdetails

6 Evaluation

6.1 Testumgebung

6.2 Testszenarien

6.3 Ergebnisse

7 Zusammenfassung und Ausblick